

Contents

RMM System — Technical Guide	3
Table of Contents	3
1. System Architecture	3
Component Diagram	3
Request Flow: Dashboard → API → Agent	4
2. Data Models	5
Model Relationships	5
Model Reference	5
3. API Endpoints	11
Auth (/api/auth/)	11
Agents (/api/agents/)	11
Devices (/api/devices/)	12
Alerts (/api/alerts/)	12
Tickets (/api/tickets/)	13
Scripts (/api/scripts/)	13
Patches (/api/patches/)	13
Automation (/api/automation/)	14
Reports (/api/reports/)	14
Billing (/api/billing/)	14
Org Settings (/api/admin/)	15
Customers (/api/customers/)	15
Users (/api/users/)	15
Admin (/api/admin/)	15
Network (/api/network/)	16
Dashboard Summary (/api/dashboard/)	16
4. Authentication & Authorization	16
JWT Flow	16
MFA Flow	18
Agent Token Flow	18
Role Permissions Matrix	18
5. Celery Tasks & Schedules	19
Beat Schedule	19
Task Signatures	19
Starting Celery (Windows)	22
6. Agent Internals	22
Main Loop	22
Task Execution	22
Patch Scanning	23
Software Collection	23
Resilience Features	24
7. Built-in Scripts Reference	24
8. Dashboard API Client	24
Session Management	25
Retry + 401 Refresh	25
Key Methods	25
HTTPS Warning	26

Cache Strategy	26
9. Performance Optimizations	27
Database	27
Celery	27
Dashboard	28
Metrics History — 7-Day Fallback	28
Responsive Design — CSS Breakpoints	29
Device Health Map — Clickable Cards	30
10. Extension Guide	30
Adding a New Dashboard Page	30
Adding a New API Route	31
Adding a New Built-in Script	31
Adding WiFi/Agentless Devices	32
Adding or Changing a Supported Currency or Timezone	33
Adding a New Report Template	33
11. Security Model	34
Network Boundaries	34
Authentication Layers	34
Known Risks & Mitigations	34
Production Checklist	35
12. Environment Variables Reference	35
13. Docker Deployment	36
docker-compose.yml (project root)	36
Dockerfiles	37
Usage	37
Environment note	37
Flask Reloader Stability (<code>use_reloader=False</code>)	37
Security hardening applied at startup (<code>api/app.py</code>)	38
14. AI Assistant	38
Architecture	38
New files	38
POST <code>/api/assistant/chat</code>	39
System prompt structure	39
Role extraction from JWT	40
Dashboard widget	40
Adding the widget to a page	41
Environment variables	41
Dependency	41
Security properties	41
Extension points	42
16. Commercial Audit — Phase F	42
F.1 Agent Multi-Customer Routing	42
F.2 DPAPI Agent Token Encryption	42
F.3 OpenAPI 3.0 / Swagger	42
F.4 Redis Cache (<code>api/utils/cache.py</code>)	42
F.5 Alert Thundering Herd Fix	42
F.6 Webhook Dispatch (<code>api/utils/webhook.py</code>)	43
F.7 Device Ownership (Client Role)	43

F.8 Script Run Audit	43
F.9 GDPR Endpoints	43
F.10 Billing Automation	43
F.11 Configurable SLA Policies	43
F.12 Database Backup (<code>tasks/backup_tasks.py</code>)	43
F.13 Migration Chain (complete)	44

RMM System — Technical Guide

Audience: Developers, system architects, and advanced administrators

Stack: Flask 3 · SQLAlchemy 2 · Celery 5 · Streamlit 1.58 · PostgreSQL 15 · Redis/Memurai

Version: 1.2 (responsive design, multi-currency/timezone, network scan fix)

Table of Contents

1. System Architecture
 2. Data Models
 3. API Endpoints
 4. Authentication & Authorization
 5. Celery Tasks & Schedules
 6. Agent Internals
 7. Built-in Scripts Reference
 8. Dashboard API Client
 9. Performance Optimizations
 10. Extension Guide
 11. Security Model
 12. Environment Variables Reference
 13. Docker Deployment
 14. AI Assistant
-

1. System Architecture

Component Diagram

```

Browser  http://localhost:8501
Streamlit Dashboard (Python)
dashboard/app.py + dashboard/pages/*.py  (16 pages)
dashboard/utils/api_client.py → RMMClient (session + retry)

```

```

HTTP REST  (JWT Bearer in Authorization header)
3-attempt retry, 0.5/1.0/2.0s backoff
401 → auto-refresh → retry once

```

```

Flask API  http://localhost:5000

```

```

api/app.py (application factory)
api/routes/*.py (13 blueprints)
api/models/*.py (13 SQLAlchemy models, 23 tables)
api/utils/builtin_scripts.py (7 built-in PS1 scripts)
api/utils/notifications.py (SMTP email, opt-in)
api/utils/invoice_pdf.py (ReportLab A4 PDF generator)
api/utils/oui.py (MAC vendor lookup, 500+ entries)
api/tasks/network_tasks.py (ICMP scan + agentless ping beat)

```

PostgreSQL	Redis (broker)	Agent API endpoints
:5432	:6379	/api/agents/*
db: rmmdb	Celery backend	X-Agent-Token auth

```

Celery Worker (--pool=solo on Windows)
tasks/alert_tasks.py - beat 60s/180s
tasks/patch_tasks.py - beat 1800s
tasks/automation_tasks.py - on demand
tasks/report_tasks.py - on demand
tasks/network_tasks.py - beat 300s

```

```

Python Agent (on each managed Windows machine)
agent/rmm_agent.py - main loop (60s heartbeat)
agent/collector.py - hardware/metrics/software/patches
agent/heartbeat.py - APIClient (register/heartbeat/tasks)
agent/executor.py - task execution engine
agent/script_runner.py - ps1/bat/py/sh execution

```

Request Flow: Dashboard → API → Agent

User clicks "Reboot" in Maintenance page

```

dashboard/pages/15_Maintenance.py
    client.reboot_device(device_id)
        POST /api/devices/<id>/reboot
            routes/devices.py: reboot_device()
                _queue_builtin_task(device_id, "reboot")
                    ScriptRun(script_id=<builtin_reboot_id>, device_id=...)
                        db.session.add + commit

(60s later)

```

```

agent polls GET /api/agents/<id>/tasks
returns [{type: "run_script", script_content: "..."}]
    executor.run(task)
        script_runner.run_ps1("Restart-Computer -Force")

```

2. Data Models

Model Relationships

```

Customer  < Device  < DeviceMetrics
           < InstalledSoftware
           < AlertRule.customer_id (optional)
           < PatchRecord
           < ScriptRun
           < ScheduledTaskRun

```

```

    < Ticket  < TicketComment
              assignee_id → User

```

```

    < PatchPolicy
    < AutomationProfile  < ScheduledTaskRun
    < Invoice
    < Report
    < NetworkScan

```

```

AlertRule  < Alert
Script     < ScriptRun
User       < AuditLog
Device     < AgentToken

```

Model Reference

User (users)

id	Integer PK
email	String(255) UNIQUE NOT NULL
password_hash	String(255) <i># bcrypt rounds=12</i>
full_name	String(255)
role	String(50) <i># superadmin admin technician viewer</i>
	<i># superadmin: highest privilege, bypasses all _requi</i>
	<i># cannot be deleted or modified via API</i>
mfa_secret	String(32)
mfa_enabled	Boolean default False
is_active	Boolean default True
must_change_password	Boolean default False server_default="false"
	<i># True → intercept login with force-change screen</i>
created_at	DateTime server_default=now()

Customer (customers)

```
id          Integer PK
name        String(255) NOT NULL
slug        String(100) UNIQUE
email       String(255)
phone       String(50)
tier        String(50) default 'standard'  # standard / premium / enterprise
notes       Text
is_active   Boolean default True
created_at  DateTime
```

Device (devices)

```
id          String/UUID PK
customer_id FK → customers
group_id    FK → device_groups (nullable)
hostname    String(255)
platform    String(50)           # windows / linux / mac
os_name     String(255)
os_version  String(100)
cpu_brand   String(255)
cpu_cores   Integer
ram_gb      Float
ip_address  String(50)           # local IP
serial_number String(255)
hardware_id String(64) UNIQUE    # SHA256(hostname+MAC)
status      String(50) default 'active'
is_online   Boolean default False
last_seen   DateTime
agent_version String(20)
display_name String(255)
created_at  DateTime
is_agentless Boolean default False NOT NULL # True = network-discovered, no agent
device_type String(50) default 'laptop' NOT NULL # laptop / desktop / mobile / server / unknown
vendor      String(255) nullable    # OUI lookup result, e.g. "Apple, Inc."
# NOTE: customer_id is nullable=True - agentless devices start unassigned
```

DeviceMetrics (device_metrics)

```
id          Integer PK
device_id   FK → devices
cpu_pct     Float
ram_pct     Float
disk_pct    Float               # primary disk
battery_pct Float (nullable)
uptime_seconds BigInteger
top_processes JSON              # [{pid, name, cpu}]
```

```
disks          JSON          # [{mountpoint, device, total_gb, used_gb, free_gb, percent}]
collected_at  DateTime      # NOTE: field is collected_at not recorded_at
```

AlertRule (alert_rules)

```
id             Integer PK
name           String(255) NOT NULL
metric         String(100)    # cpu_pct / ram_pct / disk_pct / offline
operator       String(10)     # > / < / ==
threshold      Float
severity       String(20)     # info / warning / critical
cooldown_minutes Integer default 15
is_active      Boolean default True
customer_id    FK → customers (nullable = global rule)
notification_channels JSON    # {"email": ["addr@domain.com"]}
```

Alert (alerts)

```
id             Integer PK
rule_id        FK → alert_rules
device_id      FK → devices
severity       String(20)
status         String(20) default 'open'    # open / acknowledged / resolved
message        Text
acknowledged_by FK → users (nullable)
created_at     DateTime
resolved_at    DateTime (nullable)
```

Ticket + TicketComment

Ticket

```
id             String/UUID PK
title          String(500) NOT NULL
description     Text
customer_id    FK → customers
device_id      FK → devices (nullable)
assignee_id    FK → users (nullable)      # Phase 8 addition
priority       String(20) default 'medium' # low / medium / high / critical
status         String(20) default 'open'   # open / in_progress / resolved / closed
source         String(50)                  # manual / alert / agent
created_at     DateTime
updated_at     DateTime onupdate=now()
```

TicketComment

```
id             Integer PK
ticket_id      FK → tickets
author_id      FK → users
body           Text NOT NULL
```

```
is_internal Boolean default False
created_at DateTime
```

Script + ScriptRun

Script

```
id          String/UUID PK
name        String(255) NOT NULL
description  Text           # used as __tag__ for built-in scripts
file_type   String(20)      # ps1 / bat / py / sh
content     Text NOT NULL
os_target   String(20) default 'windows'
is_builtin  Boolean default False
created_by  FK → users (nullable)
created_at  DateTime
```

ScriptRun

```
id          String/UUID PK
script_id   FK → scripts  NOT NULL
device_id   FK → devices
triggered_by FK → users (nullable)
exit_code   Integer
stdout      Text
stderr      Text
status      String(20) default 'pending' # pending / running / completed / failed / time
timeout_seconds Integer default 300
started_at  DateTime (nullable)
completed_at DateTime (nullable)
created_at  DateTime
```

PatchPolicy + PatchRecord

PatchPolicy

```
id          Integer PK
name        String(255) NOT NULL
customer_id FK → customers (nullable)
auto_approve_critical Boolean default True
auto_approve_important Boolean default False
auto_approve_security Boolean default False
reboot_behavior String(50) default 'prompt' # prompt / auto / never
maintenance_window_start String(10) # HH:MM
maintenance_window_end String(10)
excluded_software JSON # list of software name patterns to skip
is_active   Boolean default True
```

PatchRecord

```
id          Integer PK
device_id   FK → devices
```



```

policy_id    FK → patch_policies (nullable)
patch_name   String(500)
kb_id        String(50)
patch_type   String(50)    # critical / security / definition / update / feature / driver
status       String(50) default 'pending'    # pending / approved / deployed / failed
deployed_at  DateTime (nullable)
created_at   DateTime

```

AutomationProfile + ScheduledTaskRun

AutomationProfile

```

id           Integer PK
name         String(255) NOT NULL
customer_id  FK → customers (nullable)
schedule_type String(50)    # daily / weekly / monthly / once
schedule_day String(20)    # monday..sunday or day-of-month
schedule_time String(10)   # HH:MM
os_patch_config JSON       # {install_all, critical, security, definition, ...}
software_patch_config JSON # {update_all, excluded: [...]}
disk_config  JSON         # {defrag: bool, checkdisk: bool}
maintenance_config JSON   # {delete_temp, restore_point, clear_history, reboot, shutdown}
notification_emails String(500)
run_on_new_agents Boolean default False
is_active    Boolean default True
last_run_at  DateTime (nullable)
created_at   DateTime

```

ScheduledTaskRun

```

id           Integer PK
profile_id   FK → automation_profiles
device_id    FK → devices
status       String(20) default 'pending'
result_summary JSON
started_at   DateTime (nullable)
finished_at  DateTime (nullable)
created_at   DateTime

```

Report

```

id           Integer PK
name         String(255)
template_type String(100)  # device_health / patch_compliance / alert_summary / software_inventory
customer_id  FK → customers (nullable)
format       String(10) default 'csv'
file_path    String(500)   # absolute path to generated CSV in api/reports/
generated_by FK → users (nullable)
created_at   DateTime

```

IMPORTANT: to_dict() must include "file_path" for dashboard download to work

Invoice (invoices)

```
id                String/UUID PK
customer_id       FK → customers
invoice_number     String(20) UNIQUE nullable index    # INV-YYYY-NNNN, auto-generated on create
period_start       DateTime(timezone=True) NOT NULL
period_end         DateTime(timezone=True) NOT NULL
due_date           DateTime(timezone=True) nullable    # defaults to period_end + 30 days
device_count       Integer
per_device_rate    Numeric(10,2) NOT NULL
tax_rate           Numeric(5,4) default 0.0            # stored as decimal: 0.20 = 20%
notes              Text nullable                      # per-invoice custom notes
subtotal           Numeric(10,2)                      # device_count × per_device_rate
tax_amount         Numeric(10,2)                      # subtotal × tax_rate
total              Numeric(10,2)                      # subtotal + tax_amount
status             String(20) default 'draft'          # draft / sent / paid / overdue
sent_at            DateTime nullable
paid_at            DateTime nullable
created_at         DateTime
```

Sequential invoice number generation (api/routes/billing.py:_next_invoice_number()):

```
year = datetime.now(timezone.utc).year
count = Invoice.query.filter(Invoice.invoice_number.like(f"INV-{year}-%")).count()
return f"INV-{year}-{count + 1:04d}"
```

OrgSettings (org_settings) Singleton model — always id=1. Seeded at API startup by ensure_org_settings().

```
id                Integer PK default 1
company_name       String(255) default ""
company_address    Text default ""
company_email      String(255) default ""
company_phone      String(50) default ""
logo_data          Text nullable                      # base64 data URI (same pattern as user avatar)
payment_terms      String(100) default "Net 30"
bank_details       Text default ""
footer_notes       Text default "Thank you for your business!"
# White-label branding
app_name           String(100) default "RMM System"
tagline            String(200) default "Remote Monitoring & Management"
primary_color      String(7) default "#407E3C"
# Regional settings (added migration f1a2b3c4d5e6, down_revision e6f7a8b9c0d1)
currency           String(3) nullable server_default='USD' # ISO 4217 code
timezone           String(50) nullable server_default='UTC' # IANA tz name
```

logo_bytes() method decodes the base64 data URI and returns raw bytes for use in PDF generation. ensure_org_settings() is called from create_app() — safe to call multiple times (no-op

if row exists).

`to_dict()` returns `currency` (default "USD") and `timezone` (default "UTC") alongside all other fields. Dashboard reads these from `st.session_state["_org_settings"]` (loaded once per session in `require_auth()`).

3. API Endpoints

All endpoints prefixed with `/api/`. Protected by `@jwt_required()` unless noted.

Auth (`/api/auth/`)

Method	Path	Auth	Description
POST	<code>/login</code>	None	Email+password → access+refresh tokens, OR <code>{"status": "mfa_required", "mfa_token": "..."} (HTTP 200)</code> if user has <code>mfa_enabled=True</code>
POST	<code>/refresh</code>	Refresh token	Rotate access token
POST	<code>/logout</code>	JWT	Revoke token
POST	<code>/me/force-change-password</code>	JWT	Set new password + clear <code>must_change_password</code> flag
PUT	<code>/me/password</code>	JWT	Change password (current + new). Used by Profile page.
POST	<code>/mfa/setup</code>	JWT	Generate provisional TOTP secret. Returns <code>{secret, provisioning_uri}</code> . Not active until <code>/mfa/enable</code> called.
POST	<code>/mfa/enable</code>	JWT	Activate MFA. Body: <code>{code}</code> . Requires valid TOTP code.
POST	<code>/mfa/login</code>	None	Second login step. Body: <code>{mfa_token, code}</code> . Returns full JWT. Rate-limited 10/min.
POST	<code>/mfa/disable</code>	JWT	Disable MFA. Body: <code>{password}</code> . Requires current password.

Agents (`/api/agents/`)

Method	Path	Auth	Description
POST	<code>/register</code>	None (org_token)	Register new agent, get <code>device_id</code> + <code>agent_token</code> . Rate-limited: 10/min.
POST	<code>/<device_id>/health</code>	XsAgent-Token	Submit metrics, get queued tasks back
GET	<code>/<device_id>/tasks</code>	XsAgent-Token	Poll pending ScriptRun records

Method	Path	Auth	Description
POST	/<device_id>/task-agentless	X-Auth-Token	Submit ScriptRun result
PUT	/<device_id>/patches	X-Auth-Token	Submit pending Windows patches list
PUT	/<device_id>/software	X-Auth-Token	Submit installed software list

Devices (/api/devices/)

Method	Path	Auth	Description
GET	/	JWT	List devices (paginated, filterable)
GET	/<id>	JWT	Get device + latest metrics
PUT	/<id>	JWT	Update display_name, group_id
DELETE	/<id>	admin/tech	Remove device
GET	/<id>/metrics	JWT	Historical metrics (up to 168h). Row cap: .limit(5000) prevents unbounded response.
GET	/<id>/software	JWT	Installed software list
POST	/<id>/reboot	JWT	Queue reboot task
POST	/<id>/shutdown	admin/tech	Queue shutdown task
POST	/<id>/queue_task	admin/tech	Queue any built-in maintenance task
POST	/<id>/deploy_patch	admin/tech	Trigger patch deployment via Celery
GET	/platform_counts	JWT	Returns {"by_platform": { "windows": N, ...}, "agentless": N}
POST	/<id>/ping_check	admin/tech	Ping device IP, update is_online/last_seen. Only for is_agentless=True devices.

queue_task valid task_type values: clean_temp, defrag, check_disk, restore_point, clear_browser, reboot, shutdown

Alerts (/api/alerts/)

Method	Path	Auth	Description
GET	/	JWT	List alerts (filter by status, severity)
GET	/rules	JWT	List alert rules
POST	/rules	admin/tech	Create alert rule
PUT	/rules/<id>	admin/tech	Update alert rule
DELETE	/rules/<id>	admin	Delete alert rule

Method	Path	Auth	Description
POST	<code>/<id>/acknowledge</code>	JWT	Acknowledge alert
POST	<code>/<id>/resolve</code>	JWT	Resolve alert

Tickets (`/api/tickets/`)

Method	Path	Auth	Description
GET	<code>/</code>	JWT	List tickets (filter by status, priority)
POST	<code>/</code>	JWT tech/admin	Create ticket (was unguarded — any JWT; now requires technician or admin role)
GET	<code>/<id></code>	JWT	Get ticket + comments
PUT	<code>/<id></code>	JWT tech/admin	Update status, priority, assignee_id
DELETE	<code>/<id></code>	JWT admin	Delete ticket
POST	<code>/<id>/comments</code>	JWT	Add comment

Scripts (`/api/scripts/`)

Method	Path	Auth	Description
GET	<code>/</code>	JWT	List scripts (filter built-in/custom)
POST	<code>/</code>	JWT tech/admin	Create script
PUT	<code>/<id></code>	JWT tech/admin	Update script
DELETE	<code>/<id></code>	JWT admin	Delete script
POST	<code>/<id>/run</code>	JWT tech/admin	Create ScriptRun records for target devices
GET	<code>/runs</code>	JWT	List recent ScriptRun records
GET	<code>/runs/<id></code>	JWT	Get ScriptRun detail (stdout, stderr)

Patches (`/api/patches/`)

Method	Path	Auth	Description
GET	<code>/</code>	JWT	List patch records (filter by status, device)
PUT	<code>/<id>/approve</code>	JWT tech/admin	Approve a patch record
POST	<code>/approve_bulk</code>	JWT tech/admin	Approve multiple patch records
GET	<code>/policies</code>	JWT	List patch policies

Method	Path	Auth	Description
POST	/policies	JWT admin	Create patch policy
PUT	/policies/<id>	JWT admin	Update patch policy

Automation (/api/automation/)

Method	Path	Auth	Description
GET	/profiles	JWT	List automation profiles
POST	/profiles	JWT	Create profile
PUT	/profiles/<id>	JWT admin/tech	Update profile
DELETE	/profiles/<id>	JWT admin	Delete profile
POST	/profiles/<id>/run	JWT admin/tech	Trigger immediate profile run
GET	/runs	JWT	List ScheduledTaskRun records

Reports (/api/reports/)

Method	Path	Auth	Description
GET	/	JWT	List reports
POST	/	JWT admin/tech	Create report + trigger generation
GET	/<id>	JWT	Get report metadata (includes file_path)

Billing (/api/billing/)

Method	Path	Auth	Description
GET	/invoices	JWT	List invoices (optional ?customer_id=)
POST	/invoices	JWT admin	Generate invoice — auto-assigns invoice_number (INV-YYYY-NNNN), accepts tax_rate, due_date, notes
GET	/invoices/<id>	JWT	Get single invoice
PUT	/invoices/<id>/status	JWT admin	Update invoice status (draft→sent→paid/overdue)
DELETE	/invoices/<id>	JWT admin	Delete invoice
GET	/invoices/<id>/pdf	JWT	Generate and stream A4 PDF (Content-Disposition: attachment)
POST	/invoices/<id>/send_email	JWT admin	Email PDF to customer via SMTP. Returns 503 if SMTP_HOST not configured.

Org Settings (/api/admin/)

Method	Path	Auth	Description
GET	/org-settings	JWT admin	Return OrgSettings singleton (includes logo_data base64 URI, currency , timezone)
PUT	/org-settings	JWT admin	Update text fields: company name, address, email, phone, payment terms, bank details, footer notes, app_name , tagline , primary_color , currency (ISO 3-letter), timezone (IANA name)
PUT	/org-settings/logo	JWT admin	Upload logo — multipart file upload, Pillow-resized to max 400px wide, stored as base64
DELETE	/org-settings/logo	JWT admin	Remove logo (sets logo_data = null)

Customers (/api/customers/)

Method	Path	Auth	Description
GET	/	JWT	List customers
POST	/	JWT admin/tech	Create customer
GET	/ <id>	JWT	Get customer detail
PUT	/ <id>	JWT admin/tech	Update customer
DELETE	/ <id>	JWT admin	Deactivate customer

Users (/api/users/)

Method	Path	Auth	Description
GET	/	JWT admin	List users
POST	/	JWT admin	Create user
PUT	/ <id>	JWT admin	Update user (role, active, password)

Admin (/api/admin/)

Method	Path	Auth	Description
GET	/users	JWT admin	List all users
POST	/users	JWT admin	Create user — accepts must_change_password bool

Method	Path	Auth	Description
PUT	/users/<id>	JWT admin	Update user — accepts <code>must_change_password</code> bool. Returns 403 if target is superadmin.
DELETE	/users/<id>	JWT admin	Delete user. Returns 403 if target is superadmin.
GET	/org-token	JWT admin	Return <code>ORG_REGISTRATION_TOKEN</code> for display in Admin panel
GET	/server_ips	None	Returns <code>{"hostname": "...", "lan_ips": ["192.168.x.x"]}</code> via <code>socket.getaddrinfo</code>

Network (/api/network/)

Method	Path	Auth	Description
POST	/scan	JWT admin/tech	Trigger network discovery scan (dispatches <code>run_network_scan.delay</code>)
GET	/scans	JWT	List scan results
POST	/agentless_devices	JWT admin/tech	Batch upsert agentless devices. Body: <code>{"hosts": [...], "customer_id": optional}</code> . Returns <code>{"created": N, "updated": M, "skipped": K}</code> .

Dashboard Summary (/api/dashboard/)

Method	Path	Auth	Description
GET	/summary	JWT	Aggregated counts (devices, alerts, tickets, online)
GET	/health_map	JWT	Device health list (limit 500) — returned objects include <code>id</code> , <code>hostname</code> , <code>status</code> , <code>is_online</code> ; used by Dashboard Overview to render clickable cards
GET	/activity	JWT	Recent audit log entries

4. Authentication & Authorization

JWT Flow

Login (no MFA):


```
POST /api/auth/login {email, password}
→ {access_token, refresh_token, user: {id, email, role, full_name, must_change_password}}
```

Login (MFA enabled):

```
POST /api/auth/login {email, password}
→ HTTP 200 {status: "mfa_required", mfa_token: "<short-lived JWT>"}
mfa_token is a 5-min TTL JWT with claim {purpose: "mfa_pending"}.
Cannot be used for any other API call.
Client must immediately call:
POST /api/auth/mfa/login {mfa_token, code}
→ {access_token, refresh_token, user: {...}}
```

Force password change (if must_change_password=True):

```
Dashboard intercepts in app.py before showing any page.
User fills new password form → POST /api/auth/me/force-change-password {new_password}
→ clears must_change_password flag → st.rerun() → normal dashboard
```

Dashboard stores:

```
st.session_state["access_token"] = access_token
st.session_state["refresh_token"] = refresh_token
st.session_state["user"]         = user dict
st.query_params["tok"]           = access_token  ← written once at login
st.query_params["rtok"]          = refresh_token ← written once at login
(Tokens are stripped from URL by require_auth() after first restore.
Session state carries them from that point. Mitigates browser history exposure.)
```

API call:

```
Authorization: Bearer <access_token>
```

On 401:

```
POST /api/auth/refresh (Authorization: Bearer <refresh_token>)
→ {access_token}
Update session_state, retry original request once
```

On page reload (F5) or navigation:

```
Streamlit session state is wiped on browser reload.
app.py reads st.query_params["tok"] and ["rtok"] before session state check.
require_auth() in every page also re-reads and re-stamps both params.
require_auth() re-stamps ?tok= and ?rtok= on every authenticated page
load - ensures F5 on any page restores both access and refresh tokens,
allowing silent renewal even after the 900s access token TTL expires.
```

NOTE: Both access (?tok=) and refresh (?rtok=) tokens are stored in URL params. This is a pragmatic trade-off for Streamlit's stateless page reload model — without it, any F5 reload logs the user out. Mitigated by short access token TTL (900s default). In production with HTTPS, tokens in URL are not visible in transit but appear in server access logs — rotate JWT_ACCESS_TOKEN_EXPIRES to 300s for higher-security environments.

MFA Flow

Setup:

```
POST /api/auth/mfa/setup
→ {secret, provisioning_uri} (provisional -- not active yet)
```

Enable (after QR scan):

```
POST /api/auth/mfa/enable {code: "123456"}
→ {message: "MFA enabled"} -- sets mfa_enabled=True on User
```

Login with MFA:

```
POST /api/auth/login {email, password}
→ {status: "mfa_required", mfa_token: "<5-min JWT>"} (HTTP 200)
mfa_token JWT claims: {purpose: "mfa_pending", sub: user_id}
Cannot be used for any endpoint except /mfa/login.
```

```
POST /api/auth/mfa/login {mfa_token, code}
Rate-limited: 10/min
→ {access_token, refresh_token, user: {...}} (full session)
```

Disable:

```
POST /api/auth/mfa/disable {password}
→ {message: "MFA disabled"} -- requires current password for confirmation
```

Dashboard flow: - login() in dashboard/utils/auth.py returns "ok" / "mfa_required" / "error" (not bool). - On mfa_required: stores mfa_token in st.session_state["mfa_pending_token"]. - app.py route block checks mfa_pending_token before access_token — shows show_mfa_step() full-screen form. - Back button clears mfa_pending_token and returns to login.

QR code generation: qrcode[pil]==8.0 in dashboard/requirements.txt.

Agent Token Flow

Agent registers:

```
POST /api/agents/register
Body: {hardware_id, hostname, platform, os_name, cpu_brand, cpu_cores,
      ram_gb, ip_local, mac_address, org_token}
→ {device_id, agent_token}
```

Stored in: agent/agent_state.json

Subsequent requests:

```
Header: X-Agent-Token: <agent_token>
```

Token stored hashed in agent_tokens table.

On 401: agent re-registers (full registration flow).

Role Permissions Matrix

Action	viewer	technician	admin	superadmin
View dashboard, devices, alerts				
Create/update tickets	—			
Acknowledge/resolve alerts	—			
Run scripts	—			
Queue maintenance tasks	—			
Approve/deploy patches	—			
Manage automation profiles	—			
Create/update customers	—			
Generate reports	—			
Create/manage users	—	—		
Delete any resource	—	—		
View Admin page	—	—		
Manage billing/invoices	—	—		
Modify/delete superadmin account	—	—	—	— (CLI only)

NOTE: All `_require_role()` guards in all 9 route files include a superadmin bypass: `if claims.get("role") == "superadmin": return None`. The superadmin account itself cannot be modified or deleted via any API endpoint — use `api/reset_superadmin.py` for password changes.

5. Celery Tasks & Schedules

Beat Schedule

Task	Schedule	Module
<code>evaluate_all_rules</code>	Every 60s	<code>tasks.alert_tasks</code>
<code>mark_offline_devices</code>	Every 180s	<code>tasks.alert_tasks</code>
<code>sync_patch_status</code>	Every 1800s	<code>tasks.patch_tasks</code>
<code>ping_agentless_devices</code>	Every 300s	<code>tasks.network_tasks</code>

Task Signatures

`evaluate_all_rules()`

- Fetches all active alert rules
- N+1 fix:** collects all device IDs per rule batch, then loads all latest metrics in a single subquery:

```
max_id_subq = db.select(func.max(DeviceMetrics.id).label("max_id"))\
    .where(...).group_by(DeviceMetrics.device_id).subquery()
latest_by_device = {m.device_id: m for m in DeviceMetrics.query.join(max_id_subq, ...).all}
```

Previously fired one DB query per device per rule (up to 5,000 queries/cycle). Now: 1 query per rule batch regardless of device count.

- Evaluates `metric operator threshold` (e.g., `cpu_pct > 90`)

- Respects `cooldown_minutes` — no duplicate alert within cooldown
- Creates `Alert` record on breach
- Alert message now includes last-seen timestamp for offline alerts: "DESKTOP-ABC has gone offline (last seen: 2024-01-15 14:32 UTC)"
- Calls `send_alert_notification()` if `notification_channels.email` is set

`mark_offline_devices()`

- Marks devices as `is_online=False` if `last_seen > 5` minutes ago
- Creates offline alerts if a matching rule exists

`sync_patch_status()`

- Iterates all active `PatchPolicy` records
- For each policy's scope (customer or global):
 - Auto-approves `PatchRecord` rows where `status='pending'` matching policy flags
 - Respects `excluded_software` name patterns

`deploy_patches(device_id, patch_ids)`

- Looks up `PatchRecord` rows by ID
- Builds a PowerShell script using `PSWindowsUpdate` to install those KB IDs
- Creates or updates a `Script` record with tag `__deploy_patches_transient__`
- Creates a `ScriptRun` pointing to that script for the device
- Marks `PatchRecord.status = 'deployed'`

`generate_report(report_id)`

- Loads `Report` record by ID
- Dispatches to template-specific data collector:
 - `device_health` — all devices + batch-fetched latest metrics
 - `patch_compliance` — all patch records + device/policy joins
 - `alert_summary` — all alerts + rule names + device hostnames
 - `software_inventory` — all installed software + device hostnames
 - `ticket_summary` — all tickets + customer names
- Writes CSV to `api/reports/<report_id>_<name>.csv`
- Updates `report.file_path` in DB

`enqueue_profile_run(profile_id)`

- Loads `AutomationProfile`
- Queries all devices in scope (by `customer_id` or global)
- Calls `_dispatch_profile_tasks(profile, device_id, db_session)` per device
- Updates `profile.last_run_at`

`_dispatch_profile_tasks(profile, device_id, db_session)`

- Reads `disk_config` and `maintenance_config` JSON

- For each enabled task type, creates a `ScriptRun` pointing to the corresponding built-in script
- Also creates `ScriptRun` records for any custom scripts attached to the profile

`run_network_scan(scan_id)`

- Parses CIDR range from the `NetworkScan` record
- Concurrent ICMP ping sweep via `ThreadPoolExecutor(50)` — all IPs in range pinged in parallel
- For each live IP: ARP table lookup for MAC address, OUI vendor lookup via `lookup_vendor(mac)`
- Reverse DNS via `_get_hostname(ip)` runs **before** port probe so hostname is available for all fallbacks
- Detection pipeline (stops at first match):
 1. `_guess_platform(vendor)` — OUI vendor string keywords
 2. `_probe_platform(ip)` — TCP ports: 62078→iOS, 5555→Android, 2+ of (445/3389/139)→Windows (2-port threshold prevents routers/NAS with only SMB from being misidentified), 548→macOS, 22→Linux; skipped entirely for router hostnames
 3. `_guess_platform_from_hostname(rdns)` — 50+ keyword match on rDNS hostname (brand names + Samsung model numbers S10–S24, Note, A12–A73, Fold, Flip, Ultra); catches Android phones with ADB disabled
- Router/gateway hostnames (`fritz.box`, `router`, `gateway`, `modem`, etc.) are included in `discovered_hosts` for scan display but **never persisted** as agentless device records
- Calls `_upsert_agentless_host(...)` for all non-router live hosts
- Updates `NetworkScan` record with status, host count, and completion timestamp

`_upsert_agentless_host` lookup order (stops at first match): 1. MAC address — primary stable key 2. IP address across ALL devices (not just agentless) — prevents duplicates for agent-managed PCs 3. Bare hostname match (strips `.fritz.box/.local`, case-insensitive) — catches multi-adaptor machines where agent and scan see different MACs (e.g. Ethernet vs WiFi adapter) - On match with non-agentless device: returns `"skipped"` — never modifies agent-managed records - On match with agentless device: upgrades `platform/device_type` if currently “unknown” and new scan detects better info

NOTE: Phones with fully randomized MACs and all ports blocked will still reach the hostname fallback. If the router assigns a model-name hostname (Fritz!Box, ASUS, TP-Link all do), the device will be identified. Devices that remain “unknown” after all three stages can be corrected via the Edit button on the agentless device row.

Stale Network Scan Cleanup (startup, not Celery) Network scans run in a `threading.Thread(daemon=True)` — if Flask restarts mid-scan, the thread is killed but the `NetworkScan` DB record stays in `"running"` forever. On every `create_app()` call, a cleanup block runs **before** the API accepts requests:

```
cutoff = datetime.now(timezone.utc) - timedelta(minutes=5)
stale = NetworkScan.query.filter(
    NetworkScan.status == "running",
    NetworkScan.started_at < cutoff,
).all()
for s in stale:
    s.status = "failed"
```

```

        s.completed_at = datetime.now(timezone.utc)
        s.discovered_hosts = [{"error": "Scan interrupted (server restarted)"}]
db.session.commit()

```

This resolves all orphaned scans automatically on every API start. No manual DB intervention needed.

ping_agentless_devices()

- Loads all Device records where is_agentless=True
- Pings each device's ip_address
- Updates is_online=True and last_seen=now() on response
- Devices that have not responded for more than 10 minutes are marked is_online=False
- Changes committed in a single batch after all pings complete

Starting Celery (Windows)

```

# Worker (must use --pool=solo on Windows - no fork())
celery -A tasks.celery_app worker --pool=solo -l info

# Beat scheduler (separate process)
celery -A tasks.celery_app beat -l info

```

6. Agent Internals

Main Loop

```

startup
    load agent_state.json (device_id, token)
    if not registered: POST /api/agents/register → save state
    prime CPU counter: psutil.cpu_percent(interval=1)
    loop every 60s:
        collect_metrics()          (non-blocking cpu_percent(interval=None))
        send_heartbeat(metrics)
            on 401: re-register
            on connection error: exponential backoff 15s→300s
        get_tasks()
            executor.run(task) for each queued task
        if time - last_patch > patch_interval:
            get_pending_patches() (WUA COM via PowerShell)
            report_patches(patches)
        flush pending_results.json (Phase C-6 local queue)

```

Task Execution

Agent receives tasks in format:

```

{
  "type": "run_script",

```

```

    "run_id": "<uuid>",
    "script_type": "ps1",
    "script_content": "...",
    "timeout": 300
}

```

executor.py passes to script_runner.py: - **ps1**: powershell -NoProfile -NonInteractive -Command <content> with CREATE_NO_WINDOW - **bat**: cmd /c <tempfile.bat> - **py**: python <tempfile.py> - **sh**: bash <tempfile.sh> (Linux/macOS agents)

After execution, agent POSTs result to /api/agents/<id>/task_result:

```

{
  "run_id": "<uuid>",
  "exit_code": 0,
  "stdout": "...",
  "stderr": "",
  "status": "completed"
}

```

Patch Scanning

Uses PowerShell WUA COM object — requires no external package:

```

$session = New-Object -ComObject Microsoft.Update.Session
$searcher = $session.CreateUpdateSearcher()
$result = $searcher.Search("IsInstalled=0 and Type='Software'")

```

Returns up to 500 patches as [{name, kb_id, patch_type}]. Reported via PUT /api/agents/<id>/patches. API deduplicates by patch_name and creates PatchRecord rows with status='pending'.

Software Collection

get_installed_software() merges two sources, deduplicated by lowercase name:

Source	Function	Notes
Windows registry	_get_registry_software()	3 hive paths, 20s hard deadline
winget	_get_winget_software()	Skips non-ASCII lines (progress bars); finds separator line before parsing

Winget parser detail: winget list prints Unicode block-char progress bars (, ord > 127) before the real data table. The parser skips these lines, finds the first all-dash separator line (length > 10), then parses only lines after it. The dashboard adds a _clean() filter (strips U+2500–U+259F) as defence-in-depth before HTML rendering.

Resilience Features

Feature	Implementation
Non-blocking CPU	<code>cpu_percent(interval=None)</code> after startup prime
Bounded process scan	Cap at 200 processes, 3s timeout
Bounded registry scan	20s hard deadline
Winget Unicode parser	Skip <code>ord > 127</code> lines; locate separator line before data
Heartbeat backoff	15s → 30s → 60s ... → 300s cap on failure
401 re-registration	Main loop checks status code, triggers full re-register
Local result queue	<code>pending_results.json</code> , cap 100, flushed each cycle
Structured logging	<code>DeviceFilter</code> injects <code>device_id</code> into all log records

7. Built-in Scripts Reference

Built-in scripts are `Script` records with `is_builtin=True`. The `description` field stores the lookup tag. Created/updated at API startup via `ensure_builtin_scripts()`.

task_type	Tag	Script name	What it does
<code>clean_temp</code>	<code>__builtin_clean_temp</code>	Built-in: Clean Temp Files	Removes %TEMP%, %TMP%, C:\Windows\Temp
<code>defrag</code>	<code>__builtin_defrag</code>	Built-in: Defragment	<code>Optimize-Volume -DriveLetter C -Defrag</code>
<code>check_disk</code>	<code>__builtin_check_disk</code>	Built-in: Check Disk	<code>chkdsk C: /f</code> — schedules for next reboot if drive locked; exits 0 in all cases
<code>restore_point</code>	<code>__builtin_restore_point</code>	Built-in: Create Restore Point	<code>Checkpoint-Computer -Description "RMM"</code>
<code>clear_browser</code>	<code>__builtin_clear_browser</code>	Built-in: Clear Browser History	Removes Chrome/Edge/Firefox cache dirs
<code>reboot</code>	<code>__builtin_reboot</code>	Built-in: Reboot	<code>Restart-Computer -Force</code>
<code>shutdown</code>	<code>__builtin_shutdown</code>	Built-in: Shutdown	<code>Stop-Computer -Force</code>

Adding a new built-in script: 1. Add entry to `TASK_TYPE_TO_TAG` dict in `api/utils/builtin_scripts.py`
2. Add entry to `BUILTIN_SCRIPTS` dict with `name`, `content`, `file_type` 3. Restart the API — `ensure_builtin_scripts()` will create the DB record 4. Add the new `task_type` to any dispatch logic that should use it

Transient scripts (not in the built-in registry): - `__deploy_patches_transient__` — created dynamically by `deploy_patches` Celery task per deployment run

8. Dashboard API Client

`dashboard/utils/api_client.py` — `RMMClient` class

Session Management

```
# One RMMClient per browser session, stored in st.session_state
def get_client():
    if "_rmm_client" not in st.session_state:
        st.session_state["_rmm_client"] = RMMClient(...)
    return st.session_state["_rmm_client"]
```

Retry + 401 Refresh

```
def _request(self, method, path, **kwargs):
    for attempt in range(3):
        try:
            resp = self.session.request(method, url, ...)
            if resp.status_code == 401 and attempt == 0:
                self._refresh_token()
                continue
            return resp.json(), None
        except (ConnectionError, Timeout):
            time.sleep([0.5, 1.0, 2.0][attempt])
    return None, "Service unavailable"
```

Key Methods

Method	HTTP	Path
list_devices(page, per_page, **filters)	GET	/api/devices/
get_device(device_id)	GET	/api/devices/<id>
queue_device_task(device_id, task_type, timeout=300)	POST	/api/devices/<id>/queue_task
reboot_device(device_id)	POST	/api/devices/<id>/reboot
shutdown_device(device_id)	POST	/api/devices/<id>/shutdown
deploy_patches(device_id, patch_ids)	POST	/api/devices/<id>/deploy_patches
list_tickets(**filters)	GET	/api/tickets/
create_ticket(data)	POST	/api/tickets/
update_ticket(ticket_id, data)	PUT	/api/tickets/<id>
add_comment(ticket_id, body, is_internal)	POST	/api/tickets/<id>/comments
list_alerts(**filters)	GET	/api/alerts/
list_customers(per_page)	GET	/api/customers/
list_users()	GET	/api/users/
force_change_password(new_password)	POST	/api/auth/me/force-change-password
change_password(current_password, new_password)	PUT	/api/auth/me/password
mfa_setup()	POST	/api/auth/mfa/setup
mfa_enable(code)	POST	/api/auth/mfa/enable
mfa_disable(password)	POST	/api/auth/mfa/disable

Method	HTTP	Path
mfa_login(mfa_token, code)	POST	/api/auth/mfa/login
(static)		
list_patches(**filters)	GET	/api/patches/
approve_patches(patch_ids)	POST	/api/patches/approve_bulk
list_profiles()	GET	/api/automation/profiles
run_profile(profile_id)	POST	/api/automation/profiles/<id>/run
create_report(data)	POST	/api/reports/
list_reports()	GET	/api/reports/
get_platform_counts()	GET	/api/devices/platform_counts
ping_check_device(device_id)	POST	/api/devices/<id>/ping_check
upsert_agentless_devices(hosts, customer_id=None)	POST	/api/network/agentless_devices
trigger_network_scan(customer_id, scan_range)	POST	/api/network/scan
get_server_ips()	GET	/api/admin/server_ips
update_device(device_id, data)	PUT	/api/devices/<id>
delete_device(device_id)	DELETE	/api/devices/<id>
list_invoices(customer_id=None)	GET	/api/billing/invoices
get_invoice(invoice_id)	GET	/api/billing/invoices/<id>
generate_invoice(data)	POST	/api/billing/invoices
update_invoice_status(invoice_id, status)	PUT	/api/billing/invoices/<id>/status
delete_invoice(invoice_id)	DELETE	/api/billing/invoices/<id>
get_invoice_pdf_bytes(invoice_id)	GET	/api/billing/invoices/<id>/pdf
		— returns (bytes, error), bypasses JSON parsing
send_invoice_email(invoice_id)	POST	/api/billing/invoices/<id>/send-email
get_org_settings()	GET	/api/admin/org-settings
update_org_settings(data)	PUT	/api/admin/org-settings
upload_org_logo(file_bytes, content_type)	PUT	/api/admin/org-settings/logo
		— uses requests.put(files=), bypasses JSON headers
delete_org_logo()	DELETE	/api/admin/org-settings/logo

HTTPS Warning

api_client.py logs a WARNING at import time if API_BASE_URL uses http:// pointing to a non-localhost host. This warns that tokens and data will be transmitted unencrypted.

Cache Strategy

```
@st.cache_data(ttl=30)    # fast-changing: devices, alerts
@st.cache_data(ttl=60)    # medium: tickets, customers
@st.cache_data(ttl=120)   # slow-changing: scripts, users, patch policies
```

9. Performance Optimizations

Database

Optimization	Location	Detail
Composite indexes	Direct SQL (post-migration)	7 new indexes: <code>ix_device_metrics_device_collecte</code> <code>ix_devices_customer_online,</code> <code>ix_alerts_device_status,</code> <code>ix_alerts_status_triggered_at,</code> <code>ix_patch_records_device_status,</code> <code>ix_tickets_customer_status,</code> <code>ix_script_runs_device_status</code>
N+1 alert evaluation fix	<code>tasks/alert_tasks.py</code>	<code>evaluate_all_rules()</code> now loads all latest metrics per rule in 1 subquery. Was up to 5,000 queries/cycle (50 rules x 100 devices).
Metrics endpoint row cap	<code>routes/devices.py</code>	GET <code>/api/devices/<id>/metrics</code> applies <code>.limit(5000)</code> . Was unbounded (10,080+ rows for 7-day window).
Batch latest metrics	<code>routes/devices.py:_batch_latest_metrics</code>	Subquery MAX(id) per device — 1 query instead of N
Dashboard aggregation	<code>routes/dashboard.py</code>	4 DB aggregations replace 8 COUNT queries
Health map cap	<code>routes/dashboard.py</code>	<code>.limit(500)</code> prevents full-table load
N+1 agent tasks	<code>routes/agents.py</code>	Pre-fetch scripts by id dict before loop
DB connection pool	<code>config.py</code>	<code>pool_size=10,</code> <code>max_overflow=20,</code> <code>pool_pre_ping=True</code>

Celery

Optimization	Detail
<code>acks_late=True</code>	Task not acknowledged until completion — survives worker crash
<code>OperationalError</code> retry	DB connection drops retry with exponential backoff

Optimization	Detail
Automation offloaded	Device-loop in <code>enqueue_profile_run</code> runs in Celery, not in request thread

Dashboard

Optimization	Detail
RMMClient session reuse	One <code>requests.Session</code> per browser session — persistent HTTP connection
Retry backoff	<code>ConnectionError/Timeout</code> only — 0.5s/1.0s/2.0s
<code>st.cache_data</code>	TTL caching on list endpoints — prevents repeated API calls on re-renders
<code>st.spinner</code>	All data loads wrapped — user sees progress feedback
Graceful degradation	<code>st.warning</code> instead of <code>st.stop()</code> — page stays interactive on partial failure
Token URL security	<code>require_auth()</code> strips <code>?tok=/?rtok=</code> from URL after first restore. Tokens no longer re-stamped on every page load — reduces browser history exposure.
Org settings session cache	<code>require_auth()</code> loads GET <code>/api/admin/org-settings</code> once per session into <code>st.session_state["_org_settings"]</code> . All pages read <code>currency/timezone</code> from this dict — zero repeated API calls. Admin page clears <code>_org_settings</code> from session state after Regional Settings save to force a fresh load.

Metrics History — 7-Day Fallback

`dashboard/pages/04_Devices.py` — `_render_agent_row()` metrics history block.

Default window: 24 hours (GET `/api/devices/<id>/metrics?hours=24`). If the API returns an empty list (agent was offline for >24h), the dashboard retries with `hours=168` (7 days). On fallback: - An `st.info` banner shows: *"No data in last 24 h — showing last N readings (oldest ~Xh ago). Agent may be offline."* - Chart title changes from "24-hour usage history" to "7-day usage history (agent offline)". - If the 7-day window is also empty, `st.warning("No metric history available.")` is shown.

The `collected_at` timezone handling: `pd.to_datetime()` preserves tz-aware timestamps; the “hours ago” calculation converts to UTC via `.tz_convert("UTC")` before diffing against `pd.Timestamp.now(tz="UTC")`.

Responsive Design — CSS Breakpoints

Responsive styles live entirely in `dashboard/utils/styles.py` (`_GLOBAL_CSS_RULES`) and are injected as a static CSS file via `inject_css()`. The login page adds its own mobile overrides through `_login_css()` in `dashboard/app.py`.

Breakpoints:

Breakpoint	Selector	Key rules
1024px (tablet)	<code>@media (max-width: 1024px)</code>	Padding → 1.25rem, metric value → 1.6rem
768px (mobile)	<code>@media (max-width: 768px)</code>	Columns stack (<code>flex-wrap: wrap, flex: 1 1 100%</code>), 44px tap targets, 1rem input font, tabs scroll horizontally, DataFrames scroll, sidebar capped at 82vw, login card full-width
480px (small phone)	<code>@media (max-width: 480px)</code>	Padding → 0.4rem, heading/metric further reduced

Column stacking — how it works:

Streamlit renders `st.columns()` as `[data-testid="stHorizontalBlock"]` with children `[data-testid="column"]`. On mobile:

```
[data-testid="stHorizontalBlock"] { flex-wrap: wrap !important; }
[data-testid="column"]             { width: 100% !important; min-width: 100% !important; flex: 1; }
```

All column layouts — stat cards, device rows, billing columns — stack vertically. Empty spacer columns (e.g. `_`, `col`, `_` in login) have no content so they take zero visible height.

Touch targets (WCAG 2.5.5):

```
.stButton > button      { min-height: 44px !important; }
.stTextInput input      { min-height: 44px !important; font-size: 1rem !important; }
```

`font-size: 1rem` on inputs prevents iOS Safari from auto-zooming on focus.

Tab overflow:

```
[data-testid="stTabs"]      { overflow-x: auto !important; -webkit-overflow-scrolling: touch; }
[data-testid="stTabs"] [role="tablist"] { flex-wrap: nowrap !important; }
```

Tabs scroll horizontally on mobile instead of wrapping or clipping.

Login page specifics (`_login_css()` in `app.py`): Same column-stack rules are duplicated inside the login CSS block so they apply before `inject_css()` is called. Login-specific: `min-height: 48px` on inputs/buttons (slightly larger for first-touch UX), `brand-title` shrinks to 1.5rem.

Platform limitation: Streamlit injects its own `<meta name="viewport" content="width=device-width, initial-scale=1">` — no override needed. However, Streamlit renders in an iframe context where the `window.innerWidth` seen by CSS is the iframe width, not the device width. On current Chrome/Safari (mobile), this equals the viewport width correctly. On some embedded or WebView contexts, breakpoints may not fire. No workaround exists within Streamlit's current architecture.

White-label color swap compatibility: `inject_css()` does a string-replace of `#407E3C` → custom color before writing the CSS file. The responsive rules contain no color values, so the swap is unaffected.

Device Health Map — Clickable Cards

Cards in `dashboard/pages/01_Dashboard.py` use `st.button()` + `st.switch_page()` for reliable Streamlit-native navigation. Plain `<a href>` links and `components.html()` with `window.parent.location.href` are intercepted/sandboxed by Streamlit's SPA router and do not work.

Navigation flow: 1. User clicks a health map card → `st.session_state["_nav_device"] = str(device_id)` → `st.switch_page("pages/04_Devices.py")` 2. `04_Devices.py` reads `st.session_state.pop("_nav_device", None)` on load, falls back to `st.query_params.get("device")` for direct URL access 3. If matched, pre-fills search box with the device hostname and shows an info banner

CSS scoping: Card button styles are injected inside the `else` branch of the health map render and use `section[data-testid="stMain"] button[kind="secondary"]` to target only main-area secondary buttons — sidebar buttons are excluded. Refresh button uses `type="primary"` to opt out of card styling entirely.

10. Extension Guide

Adding a New Dashboard Page

1. Create `dashboard/pages/NN_PageName.py`
2. Follow the standard page template:

```
import streamlit as st
from utils.auth import require_auth
from utils.nav import render_sidebar
from utils.styles import inject_css

st.set_page_config(page_title="Page Name - RMM", layout="wide")
inject_css()

client = require_auth()
render_sidebar()
```

```
st.markdown('<h1 style="margin:0">Page Name</h1>', unsafe_allow_html=True)
```

```
with st.spinner("Loading..."):
    data, err = client.some_method()
if err:
    st.warning(f"Could not load data - {err}")
    st.stop()
```

3. Add a nav link in dashboard/utils/nav.py under the appropriate section

Adding a New API Route

1. Create api/routes/mymodule.py:

```
from flask import Blueprint, request, jsonify
from flask_jwt_extended import jwt_required, get_jwt
from extensions import db
```

```
mymodule_bp = Blueprint("mymodule", __name__)
```

```
@mymodule_bp.route("/", methods=["GET"])
@jwt_required()
def list_items():
    ...
    return jsonify({"items": [...]}), 200
```

2. Register in api/app.py:

```
from routes import mymodule
app.register_blueprint(mymodule.mymodule_bp, url_prefix='/api/mymodule')
```

3. Add corresponding method to dashboard/utils/api_client.py

Adding a New Built-in Script

1. Edit api/utils/builtin_scripts.py:

```
# Add to TASK_TYPE_TO_TAG:
"my_task": "__builtin_my_task__",

# Add to BUILTIN_SCRIPTS:
"my_task": {
    "name": "Built-in: My Task",
    "content": "# PowerShell script here\nWrite-Output 'Done'",
    "file_type": "ps1",
},
```

2. Restart the API — ensure_builtin_scripts() creates the DB record automatically
3. Optionally add the task_type to _dispatch_profile_tasks() in tasks/automation_tasks.py
4. Optionally add a UI button in the relevant dashboard page calling client.queue_device_task(device_id, "my_task")

Adding WiFi/Agentless Devices

For Windows/Linux/macOS laptops on the same WiFi/LAN:

1. Find the server's LAN IP: Admin → System Info → Server IP Addresses, or check your router.
2. Copy the `agent/` folder to the target machine (USB, network share, or git clone).
3. On the target machine, install dependencies:

```
cd agent
python -m venv venv
venv\Scripts\activate # Windows
pip install -r requirements.txt
```

4. Run the setup helper — patches `config.ini` automatically:

```
python setup_agent.py 192.168.x.x <org_token>
```

The org token is shown in Admin → System Info → Agent Enrollment Token.

5. Start the agent:

```
python rmm_agent.py
```

6. The device appears in Devices → Windows/Linux/macOS tab within 60 seconds.

For phones, tablets, and IoT devices (no agent possible):

1. Go to Network Discovery in the dashboard.
2. Enter the subnet in CIDR notation (e.g. `192.168.1.0/24`) and optionally assign a customer.
3. Click **Scan Network** — the scan takes 15–30 seconds via concurrent ICMP ping.
4. Discovered devices appear with IP, MAC, vendor (from OUI lookup), and detected platform.
 - iOS devices are identified by Apple OUI prefixes, or by port 62078 being open (port probe fallback).
 - Android devices are identified by Samsung/Google/OnePlus/etc. OUI prefixes, or by port 5555 being open.
 - When OUI lookup returns unknown, `_probe_platform(ip)` tries 6 TCP ports to detect the OS.
 - Reverse DNS (`_get_hostname`) populates a friendly hostname (e.g. `Printer.local`) when available.
5. Click **Save All to Devices** to persist them to the database.
6. Devices appear in Devices → Android/iOS/Agentless tabs.
7. Celery beat pings them every 5 minutes — online/offline status updates within 10 minutes of disconnect.

Identifying devices that evade automated detection (randomized MAC + closed ports):

Modern phones randomize their MAC address per network, defeating OUI lookup. They also typically block all probed ports. These devices will remain platform “unknown” after a scan. Use the **Edit** button on the agentless device row in the Devices page to manually set hostname, platform, and device_type. This calls `PUT /api/devices/<id>` via `client.update_device(device_id, data)`.

Adding or Changing a Supported Currency or Timezone

All supported currencies and timezones are defined as Python lists in `dashboard/utils/formatters.py`:

```
SUPPORTED_CURRENCIES = ["USD", "EUR", "GBP", "CHF", "CAD", "AUD", "JPY", "SEK", "NOK", "DKK"]

CURRENCY_SYMBOLS = {
    "USD": "$", "EUR": "€", "GBP": "£", "CHF": "Fr",
    "CAD": "CA$", "AUD": "A$", "JPY": "¥",
    "SEK": "kr", "NOK": "kr", "DKK": "kr",
}

SUPPORTED_TIMEZONES = [
    "UTC", "Europe/Berlin", "Europe/London", ...
]
```

To add a new currency: append the ISO code to `SUPPORTED_CURRENCIES` and add its symbol to `CURRENCY_SYMBOLS`. No DB migration needed — currency/timezone values are stored as plain strings.

To add a new timezone: append the IANA name to `SUPPORTED_TIMEZONES`. Verify Python's `zoneinfo` module recognizes it: `from zoneinfo import ZoneInfo; ZoneInfo("Your/Zone")` (raises `ZoneInfoNotFoundError` if invalid).

`fmt_currency(amount, currency)` — formats a numeric amount with the correct symbol:

```
fmt_currency(49.99, "EUR") # → "€49.99"
fmt_currency(49.99, "JPY") # → "¥49.99"
```

`fmt_datetime_tz(iso_str, tz_name)` — converts a UTC ISO string to local time for display:

```
fmt_datetime_tz("2026-06-16T10:00:00Z", "Europe/Berlin") # → "2026-06-16 12:00"
fmt_datetime_tz("2026-06-16T10:00:00Z", "America/New_York") # → "2026-06-16 06:00"
```

Falls back to `fmt_datetime(iso_str)` (UTC, no conversion) if `zoneinfo` raises or the `tz_name` is invalid. This prevents crashes when an unsupported timezone is stored in the DB.

Consuming regional settings in a new dashboard page:

```
_org = st.session_state.get("_org_settings", {})
_currency = _org.get("currency", "USD")
_tz       = _org.get("timezone", "UTC")
```

Then use:

```
fmt_currency(invoice["total"], _currency)
fmt_datetime_tz(invoice["created_at"], _tz)
```

`_org_settings` is loaded once per session by `require_auth()` and is always available by the time any page body runs.

Adding a New Report Template

1. Add to `tasks/report_tasks.py`:

```
def _my_template(report, db_session):
    rows = db_session.query(MyModel).all()
    return [{"col1": r.field1, "col2": r.field2} for r in rows]
```

2. Register in the dispatcher dict inside `generate_report()`:

```
COLLECTORS = {
    ...
    "my_template": _my_template,
}
```

3. Add option to the template dropdown in `dashboard/pages/08_Reports.py`

11. Security Model

Network Boundaries

Surface	Exposure	Notes
Streamlit :8501	LAN/localhost only	No direct user data — proxies to API
Flask API :5000	LAN/localhost only	All auth enforced via JWT
PostgreSQL :5432	localhost only	<code>rmm_app</code> user has no superuser
Redis :6379	localhost only	No password in dev (add one for production)

Authentication Layers

1. **Dashboard users** — JWT (HS256, `JWT_SECRET_KEY`), 900s access / 7d refresh
2. **Agents** — `X-Agent-Token` header, token stored as SHA256 hash in `agent_tokens` table
3. **API startup** — `ORG_REGISTRATION_TOKEN` required for agent registration
4. **Superadmin** — seeded at every API startup via `ensure_superadmin()` in `api/utils/superadmin.py`; bypasses all `_require_role()` checks; cannot be deleted or modified via API; password reset via `api/reset_superadmin.py` CLI

Known Risks & Mitigations

Risk	Mitigation
JWT token theft	Short expiry (900s). HTTPS in production. Tokens written to URL once at login, then stripped by <code>require_auth()</code> after first restore — no longer re-stamped on every page load.
Script injection	Scripts stored as plain text — reviewed before running. Agent now uses <code>-ExecutionPolicy RemoteSigned</code> instead of <code>Bypass</code> — locally created scripts run, remote scripts must be signed.

Risk	Mitigation
Error response leakage	400/422 handlers no longer return <code>str(e)</code> to client. Detail logged server-side only; generic message returned to caller.
CORS wildcard	<code>CORS_ORIGINS</code> env var (comma-separated) replaces <code>origins="*"</code> . Default: <code>http://localhost:8501</code> . Set to dashboard URL(s) in production.
Privilege escalation	Role enforced at API level via <code>get_jwt()</code> claims check on every mutation endpoint
Celery task injection	Tasks only dispatched by authenticated API endpoints or beat schedule
DB credentials in env	<code>.env</code> in <code>.gitignore</code> . Never committed.
Report file exposure	<code>api/reports/</code> not served via HTTP. Download reads bytes directly in dashboard.
SMTP credential exposure	<code>SMTP_PASS</code> read from env only. Never logged.
Superadmin lockout	Auto-seeded at every API startup — account always exists. Emergency password reset: <code>cd api ; python reset_superuser.py <new_password></code>

Production Checklist

- ☐ Set `FLASK_DEBUG=0`
- ☐ Use HTTPS (nginx reverse proxy with Let's Encrypt)
- ☐ Add Redis password: `requirepass <password>` in Memurai config
- ☐ Rotate `ORG_REGISTRATION_TOKEN` after all agents registered
- ☐ Change default admin password from `Admin1234!`
- ☐ Set up automated PostgreSQL backups
- ☐ Move `api/reports/` to a dedicated volume outside the project root
- ☐ Run agent as a dedicated low-privilege Windows service account (not SYSTEM)
- ☐ Enable MFA for all admin accounts (full TOTP implementation: setup, enable, login, disable via Profile page)
- ☐ Set `CORS_ORIGINS` to dashboard URL (replaces wildcard `origins="*"`)
- ☐ Set `SUPERADMIN_PASSWORD` in `.env` (now required – API will not start without it)

12. Environment Variables Reference

Variable	Required	Default	Description
<code>DATABASE_URL</code>		—	PostgreSQL connection string
<code>SECRET_KEY</code>		—	Flask secret key (32-byte hex)
<code>JWT_SECRET_KEY</code>		—	JWT signing key (32-byte hex)
<code>ORG_REGISTRATION_TOKEN</code>		—	Agent registration shared secret

Variable	Required	Default	Description
JWT_ACCESS_TOKEN_EXPIRES		900	Access token TTL in seconds
JWT_REFRESH_TOKEN_EXPIRES		604800	Refresh token TTL in seconds
REDIS_URL	—	redis://localhost:6379/0	Redis connection URL
CELERY_BROKER_URL	—	redis://localhost:6379/0	CELERY broker
CELERY_RESULT_BACKEND	—	redis://localhost:6379/1	CELERY result backend
API_BASE_URL	—	http://localhost:5000	Used by dashboard to reach API
SMTP_HOST	—	—	SMTP server (omit to disable email alerts)
SMTP_PORT	—	587	SMTP port
SMTP_USER	—	—	SMTP login username
SMTP_PASS	—	—	SMTP login password (use app-specific password)
SMTP_FROM	—	SMTP_USER	From address for alert emails
SUPERADMIN_EMAIL	—	superadmin@rmm.local	Email for the auto-seeded superadmin account
SUPERADMIN_PASSWORD		—	Now required at startup. Min 10 chars. API raises <code>RuntimeError</code> if missing or too short.
CORS_ORIGINS	—	http://localhost:8001	Comma-separated allowed CORS origins. Set to dashboard URL(s) in production.
DB_PASSWORD	—	changeme	PostgreSQL password used in <code>docker-compose.yml</code> only.
FLASK_ENV	—	production	Set to <code>development</code> for debug mode
FLASK_DEBUG	—	0	Set to 1 only in development

Generate secrets:

```
python -c "import secrets; print(secrets.token_hex(32))" # SECRET_KEY, JWT_SECRET_KEY
python -c "import secrets; print(secrets.token_hex(24))" # ORG_REGISTRATION_TOKEN
```

13. Docker Deployment

`docker-compose.yml` (project root)

6 services:

Service	Image	Command
db	postgres:16-alpine	Postgres with health check
redis	redis:7-alpine	Redis with health check
api	Built from api/Dockerfile	flask db upgrade && python app.py
celery_worker	Same as api	celery -A tasks.celery_app worker --pool=solo -l info
celery_beat	Same as api	celery -A tasks.celery_app beat -l info
dashboard	Built from dashboard/Dockerfile	streamlit run app.py

Dockerfiles

- api/Dockerfile — python:3.11-slim + libpq-dev gcc (for psycopg2 compile)
- dashboard/Dockerfile — python:3.11-slim

Usage

Start all services

`docker-compose up -d`

View API logs

`docker-compose logs -f api`

Stop (preserves data volume)

`docker-compose down`

Stop + delete all data

`docker-compose down -v`

Rebuild after code changes

`docker-compose build api dashboard && docker-compose up -d`

Environment note

When using Docker Compose, DATABASE_URL must use @db:5432 (service name), not @localhost:5432. REDIS_URL / CELERY_BROKER_URL must use redis://redis:6379/.... SUPERADMIN_PASSWORD is required — API container will exit immediately if missing.

Flask Reloader Stability (use_reloader=False)

`app.run()` sets `use_reloader=False` permanently:

```
app.run(
    host=os.getenv("API_HOST", "0.0.0.0"),
    port=int(os.getenv("API_PORT", 5000)),
    debug=os.getenv("FLASK_DEBUG", "1") == "1",
```

```

    use_reloader=False,
)

```

Why: Flask's default `watchdog` reloader monitors all files reachable via `sys.path`, including Redis/Memurai's site-packages (`resp2.py`, `resp3.py`, etc.). Memurai writes to these files during normal operation, causing Flask to restart every ~30 seconds — killing in-flight network scan threads, resetting all connections, and returning 500/connection-reset errors to the dashboard.

`use_reloader=False` eliminates these restarts. Code changes require a manual API restart during development. In production, this is the expected behavior regardless.

Security hardening applied at startup (`api/app.py`)

`_validate_env()` runs at import time and raises `RuntimeError` if:

- `SECRET_KEY` or `JWT_SECRET_KEY` < 32 characters
- `SUPERADMIN_PASSWORD` unset or < 10 characters
- `ORG_REGISTRATION_TOKEN` is the placeholder value "replace-with-a-unique-org-token"

Additional hardening:

- `X-Request-ID` header: `before_request` generates UUID if not present; echoed on all responses.
- Dev mode warning: API logs `WARNING` on startup if `FLASK_ENV != "production"`.
- CORS restricted via `CORS_ORIGINS` env var (replaces `origins="*"`).

14. AI Assistant

Architecture

```

[Streamlit page]
    render_ai_assistant(page_name, context)

[dashboard/utils/ai_assistant.py]
    POST /api/assistant/chat (JWT Bearer)

[api/routes/assistant.py]
    _build_system_prompt(role, page, context)
    anthropic.Anthropic(api_key=...).messages.create(...)

[Anthropic API - claude-haiku-4-5-20251001]
    reply text + suggested_actions list

[Streamlit sidebar chat panel]

```

New files

File	Role
<code>api/routes/assistant.py</code>	Flask blueprint, chat endpoint, system prompt builder
<code>dashboard/utils/ai_assistant.py</code>	Streamlit sidebar widget

POST /api/assistant/chat

Auth: JWT Bearer (same as all other API endpoints)

Rate limit: 30 requests per minute per IP (Flask-Limiter)

Request body:

```
{
  "message": "How do I deploy patches?",
  "page": "OS Patches",
  "context": {"pending_patches": "12", "selected_device": "DESKTOP-01"},
  "history": [{"role": "user", "content": "..."}, {"role": "assistant", "content": "..."}]
}
```

Response:

```
{
  "reply": "To deploy patches: 1. Select a device...",
  "suggested_actions": ["Deploy OS patches", "Check patch status", "What devices need updates?"]
}
```

Constraints: - message max 1000 chars (truncated server-side) - history capped at last 10 turns
- context keys/values sanitised: key max 60 chars, value max 300 chars - page max 60 chars

Error responses:

Status	Condition
400	message empty
429	Rate limit exceeded
503	AI_ASSISTANT_ENABLED=false, or ANTHROPIC_API_KEY missing/invalid, or Anthropic API down

System prompt structure

_build_system_prompt(role, page, context) assembles:

You are the AI Assistant embedded in Remote Management System (RMS)...

USER ROLE: {role}

ROLE PERMISSIONS: {role_desc}

CURRENT PAGE: {page}

PAGE PURPOSE: {page_desc}

LIVE CONTEXT:

- {key}: {value}

...

RESPONSE RULES:

1. Concise - 2-4 sentences for simple answers; numbered steps for how-to.

2. Use exact UI element names (button labels, tab names, headings).
3. Only describe features that exist. Never invent pages, buttons, or endpoints.
4. Respect role - if feature requires higher role, say so.
5. For viewers: never suggest create/edit/delete/run actions.
6. Off-topic: "I can only help with navigating this system."
7. Plain language - user may be new to RMM tools.
8. Multi-step tasks: number the steps.
9. If context shows a problem, mention the most urgent action first.

Role extraction from JWT

```
from flask_jwt_extended import jwt_required, get_jwt

@assistant_bp.route("/chat", methods=["POST"])
@jwt_required()
def chat():
    role = get_jwt().get("role", "viewer")    # role is in additional_claims
    ...
```

Critical: `get_jwt_identity()` returns the user ID (int), not the role. Role lives in `additional_claims`. Always use `get_jwt()`.

Dashboard widget

`render_ai_assistant(page_name, context)` in `dashboard/utils/ai_assistant.py`:

```
from utils.api_client import RMMClient

def render_ai_assistant(page_name="Overview", context=None):
    token = st.session_state.get("access_token")
    if not token:
        return    # not logged in - silently skip
    client = RMMClient(
        access_token=token,
        refresh_token=st.session_state.get("refresh_token", ""),
    )
    # ... initialise session state, render toggle, chat history, form, suggestions
```

Critical: Do NOT use `st.session_state.get("_rmm_client")` — that key is never set. `require_auth()` returns the client but does not store it. Build `RMMClient` directly from `access_token`.

Session state keys:

Key	Type	Purpose
<code>_ai_history</code>	<code>list[dict]</code>	Chat turns <code>{role, content}</code> — capped display at 14
<code>_ai_open</code>	<code>bool</code>	Panel open/closed
<code>_ai_seen_onboard</code>	<code>bool</code>	Whether welcome message has been shown this session

Key	Type	Purpose
<code>_ai_suggested</code>	<code>list[str]</code>	Quick action button labels from last API response

Page key: widget keys are scoped to `page_name.lower().replace(" ", "_").replace("/", "_")` to prevent duplicate-key errors across pages.

Adding the widget to a page

```
from utils.ai_assistant import render_ai_assistant

# At the bottom of the page (after all page content):
render_ai_assistant("Page Name")

# With live context (data-heavy pages):
render_ai_assistant("Overview", {
    "total_devices": total,
    "online_devices": online,
    "open_alerts": alerts,
})
```

The widget renders inside with `st.sidebar`: — no layout columns needed.

Environment variables

Variable	Required	Default	Description
<code>ANTHROPIC_API_KEY</code>	Yes	—	Anthropic API key (<code>sk-ant-api03-...</code>)
<code>AI_ASSISTANT_MODEL</code>	No	<code>claude-haiku-4-5-2024-10-11</code>	Model ID
<code>AI_ASSISTANT_ENABLED</code>	No	<code>true</code>	Set <code>false</code> to disable without removing code

Dependency

```
# api/requirements.txt
anthropic>=0.40.0
```

Install: `pip install anthropic>=0.40.0`

Security properties

- API key stored server-side only (`api/.env`) — never sent to dashboard or browser
- JWT required on `/api/assistant/chat` — unauthenticated requests return 401
- Rate limiting prevents API cost abuse (30 req/min per IP via Flask-Limiter)
- Conversation history stored in Streamlit session state only — cleared on logout/refresh, never persisted to DB
- Context values sanitised before inclusion in system prompt

Extension points

Add a new page description: Add an entry to `_PAGE_INFO` in `api/routes/assistant.py`. Key must match the `page_name` string passed from the dashboard page.

Add quick actions for a page: Add an entry to `_PAGE_ACTIONS` with the same key.

Add role-aware behaviour: Update `_ROLE_CAPABILITIES` dict.

“Do it for me” actions (future): The widget already holds a `RMMClient` instance. Wire suggested-action buttons to call `client.create_ticket(...)` etc. instead of just sending a chat message.

Voice input (future): Add `st.audio_input()` → transcribe → feed transcript to `_send_message()`. No backend changes needed. - 400/422 error handlers: detail logged server-side only, generic message returned to client.

16. Commercial Audit — Phase F

F.1 Agent Multi-Customer Routing

`api/schemas/agents.py` — `customer_id = fields.String(load_default="", validate=validate.Length(m`

`api/routes/agents.py` registration — if `customer_id` in payload and valid UUID: assigns agent to that customer. Otherwise falls back to `Customer.query.filter_by(is_active=True).first()` (backward compatible).

`agent/config.ini` — `customer_id = under [api]`. Set via `setup_agent.py <server_ip> <org_token> <customer_id>`.

F.2 DPAPI Agent Token Encryption

`agent/rmm_agent.py` — `_protect_token()` / `_unprotect_token()`. Windows: encrypts with `win32crypt.CryptProtectData`, stores as `DPAPI:<base64>`. Non-Windows or error: stores plaintext. Existing plaintext tokens auto-migrate on first write. Requires `pywin32` on Windows.

F.3 OpenAPI 3.0 / Swagger

GET `/api/docs` — SwaggerUI (CDN, brand topbar). GET `/api/openapi.json` — raw spec (47 paths, 16 tags, BearerAuth). No new pip deps.

F.4 Redis Cache (`api/utils/cache.py`)

`cache_get(key)` / `cache_set(key, value, ttl)` / `cache_delete(key)`. JSON-serialized Redis values with TTL. Silent no-op on Redis unavailability.

Applied: dashboard summary (30s, per-tenant key), `platform_counts` (60s).

F.5 Alert Thundering Herd Fix

Redis SET NX EX 55 lock in `evaluate_all_rules`. Lock key: `rmm:lock:evaluate_all_rules`. If lock not acquired, task exits. Released in `finally`.

F.6 Webhook Dispatch (`api/utils/webhook.py`)

`dispatch_alert_webhooks(channels, rule_name, device_hostname, message, severity)`.

Formats: Slack Block Kit (color-coded), MS Teams MessageCard, generic JSON POST. 5s timeout. Never raises.

F.7 Device Ownership (Client Role)

- `POST /api/terminal/sessions` — returns 404 if `device.customer_id != claims["customer_id"]`
- `POST /api/scripts/<id>/run` — filters `device_ids` to customer-scoped query; unauthorized IDs skipped

F.8 Script Run Audit

`run_script()` emits `AuditLog(action="script_run", resource_type="script", payload={script_name, device_count, device_ids})` after commit.

F.9 GDPR Endpoints

Endpoint	Behaviour
<code>GET /api/admin/users/<id>/gdpr-export</code>	Returns <code>{user, audit_log[1000], ticket_comments[1000]}</code>
<code>DELETE /api/admin/users/<id>/gdpr-delete</code>	Anonymizes PII, scrubs IPs from audit log, nulls comment author email. Irreversible.

Both admin-only. Both emit `AuditLog` entry.

F.10 Billing Automation

Customer gains `billing_day`, `per_device_rate`, `tax_rate` (migration `j1k2l3m4n5o6`).

`tasks/billing_tasks.py generate_recurring_invoices` (beat 86400s): generates draft invoice for prior month for customers where `billing_day == today` and `per_device_rate > 0`. Idempotent on `(customer_id, period_start)`.

F.11 Configurable SLA Policies

`SLAPolicy` model: `(customer_id nullable, priority)` unique. Seeded with 4 global defaults.

Routes at `/api/sla-policies/`. `tickets.py _sla_resolution_hours()` lookup: customer-specific → global → hardcoded.

F.12 Database Backup (`tasks/backup_tasks.py`)

`backup_database` (beat 86400s): `pg_dump` → `gzip` → `BACKUP_DIR`. Prunes files older than `BACKUP_RETAIN_DAYS`. Retry on timeout (3600s) or error (1800s).

Env: `BACKUP_DIR` (default `../backups`), `BACKUP_RETAIN_DAYS` (default 7).

Restore: `gunzip -c backups/rmddb_YYYYMMDD_HHMMSS.sql.gz | psql -U rmm_app rmddb`

F.13 Migration Chain (complete)

d746c05420bd → 2c8f1d3e9a4b → 93baa3927b0c → f3e2d1c0b9a8 → a1b2c3d4e5f6
→ e5d4c3b2a1f0 → b5c4d3e2f1a0 → c1d2e3f4a5b6 → d2e3f4a5b6c7 → e6f7a8b9c0d1
→ f1a2b3c4d5e6 → a2b3c4d5e6f7 → b3c4d5e6f7a8 → c4d5e6f7a8b9 → d5e6f7a8b9c0
→ f7a8b9c0d1e2 → h9i0j1k2l3m4 → i0j1k2l3m4n5 → j1k2l3m4n5o6 → k2l3m4n5o6p7